

Assignment 2 - Answers

Question 1.1

Q: Let us define the L11 language as L1 excluding the special form 'define'. Is there a program in L1 which cannot be transformed to an equivalent program in L11?

A: No. In L1 there is no `lambda` and no `if`, so `define` can only bind a variable to a value built from literals and primitive operations. Every `define` can be eliminated by inlining — substituting the bound value wherever the variable is referenced. Since L1 has no recursion, no higher-order functions, and no self-reference, every chain of defines can be flattened by substitution.

Example:

```
(L1
  (define x 5)
  (define y (+ x 1))
  (+ x y))
```

Transforms to the equivalent L11 program:

```
(L11
  (+ 5 (+ 5 1)))
```

Question 1.2

Q: Let us define the L21 language as L2 excluding the special form 'define'. Is there a program in L2 which cannot be transformed to an equivalent program in L21?

A: No. L2 adds `lambda` and `if` to L1, and `lambda` allows us to replace every `define` with an immediately-applied `lambda`. The transformation is: `(define x e) rest...` becomes `((lambda (x) rest...) e)`. Sequential defines become nested lambdas. Additionally, `define` in L2 does not support recursion — the closure captures the environment at creation time, before the binding is added — so no expressive power is lost.

Example:

```
(L2
  (define f (lambda (x) (+ x 1))))
```

```
(define y (f 5))  
(+ y 1))
```

Transforms to the equivalent L21 program:

```
(L21  
  ((lambda (f) ((lambda (y) (+ y 1)) (f 5))) (lambda (x) (+ x 1))))
```

Question 1.3

Q: Let us define the L22 language as L2, where procedures (lambda) can only have one parameter and a body with one expression. Is there a program in L2 which cannot be transformed to an equivalent program in L22?

A: No. Every L2 program can be transformed to L22 using the following transformations:

- I **Multiple parameters** → **currying**: (lambda (x y) body) becomes (lambda (x) (lambda (y) body)), and application (f 3 4) becomes ((f 3) 4).
- II **Zero parameters** → **dummy parameter**: (lambda () body) becomes (lambda (_) body), and application (f) becomes (f #f).
- III **Multiple body expressions** → **chaining with lambda**: (lambda (x) e1 e2) becomes (lambda (x) ((lambda (_) e2) e1)). This evaluates e1, discards its result via _, then evaluates e2, preserving evaluation order.

Example:

```
(lambda (x y) (+ x 1) (+ x y))
```

Transforms to the equivalent L22 expression:

```
(lambda (x) (lambda (y) ((lambda (_) (+ x y)) (+ x 1))))
```

Question 1.4

Q: Let us define the L23 language as L2, where procedures (lambda) are first-order, i.e. cannot get functions as arguments. Is there a program in L2 which cannot be transformed to an equivalent program in L23?

A: No. L23 still has `lambda`, `if`, `define`, and recursion, so it remains Turing-Complete and can compute any computable function. Higher-order functions are a syntactic convenience, not a computational necessity. For any L2 program that passes a function as an argument, we can eliminate the higher-order usage by inlining the specific function at each call site. Since L2 programs are finite, there are always finitely many functions being passed, so this transformation (defunctionalization) always terminates.

Example:

```
;; L2 (higher-order)
(define apply-twice (lambda (f x) (f (f x))))
(define square (lambda (x) (* x x)))
(apply-twice square 3)
```

Transforms to the equivalent L23 program:

```
;; L23 (first-order) – inline the specific function
(define square (lambda (x) (* x x)))
(square (square 3))
```

Question 1.5

Q: Why are special forms required in programming languages? Why can't we simply define them as primitive operators? Give an example.

A: Special forms are required because they need to control **which sub-expressions get evaluated and when**, which contradicts the default evaluation rule for procedure application (evaluate all arguments first, then apply).

If special forms were primitive operators, all their arguments would be evaluated before application, which leads to incorrect behavior. For example, `if` must evaluate only one of its two branches based on the condition — not both.

Example:

```
(define x 0)
(if (= x 0) 1 (/ 1 x))
```

If `if` were a primitive operator, the interpreter would evaluate all three arguments before applying:

| `(= x 0) → #t`

II $1 \rightarrow 1$

III $(/ 1 x) \rightarrow$ **division by zero error!**

As a special form, `if` first evaluates the condition $(= x 0)$ to `#t`, and then evaluates **only** the consequent `1`, never reaching the problematic $(/ 1 x)$.

Similarly, `define` in $(\text{define } y 5)$ must not evaluate `y` (it has no value yet), and `lambda` in $(\text{lambda } (x) (* x x))$ must not evaluate its body at definition time.

Question 1.6

Q: Is a function-body with multiple expressions required in a pure functional programming? In which type of languages is it useful? What about L3?

A: No, multiple expressions in a function body are not required in pure functional programming. In a pure functional language there are no side effects, so only the last expression's value matters — the preceding expressions have no observable effect. Any multi-expression body can be rewritten using nested lambdas, as shown in Q1.3: $(\text{lambda } (x) e1 e2)$ becomes $(\text{lambda } (x) ((\text{lambda } (_) e2) e1))$.

Multiple body expressions are useful in **imperative languages** that have side effects (e.g., mutation, I/O, printing). In such languages, each expression in the body may perform a meaningful action beyond computing a return value.

As for L3 — it is a pure functional language (no mutation, no I/O side effects), so multiple body expressions are not required. They are supported as syntactic convenience, but carry no additional expressive power.

Question 1.7

Q: What is lexical address? Give an example which demonstrates this concept.

A: A lexical address replaces a variable reference with a pair of coordinates that point to where the variable is defined:

- **Depth** — how many scoping levels up from the current scope to the scope where the variable is defined.
- **Position** — the index of the variable within that scope's parameter list.

Variables not defined in any enclosing scope are marked as free.

Example:

```
(lambda (x y)
  (
    (lambda (x) (+ x y))
    (+ x z)
  ))
```

After adding lexical addresses:

```
(lambda (x y)
  (
    (lambda (x) (+ [x : 0 0] [y : 1 1]))
    (+ [x : 0 0] [z : free])
  ))
```

- x inside the inner lambda $\rightarrow [x : 0\ 0]$: depth 0 (defined in current lambda), position 0 (first parameter).
- y inside the inner lambda $\rightarrow [y : 1\ 1]$: depth 1 (one level up), position 1 (second parameter of outer lambda).
- x in $(+ x z)$ $\rightarrow [x : 0\ 0]$: depth 0 (defined in the outer lambda), position 0.
- $z \rightarrow [z : \text{free}]$: not defined in any enclosing lambda.

Lexical addresses make variable lookup unambiguous without relying on names, which is especially useful when different scopes reuse the same variable name (like x above).

Question 1.8

Q1.8 In practical session 5, we deal with two representations of primitive operations: PrimOp and Closure. List an advantage for each of the two methods [2 points].

A: An advantage of PrimOp is that it doesn't need an environment, because its behavior is fixed. (Hence it carries no environment). Therefore, it is more efficient. An advantage of Closure is its flexibility, because a closure captures its defining environment, the same function body can behave differently depending on where it was created. This enables higher-order languages.

Question 1.9

Q1.9 a. What are the reasons that would justify switching from applicative order to normal order evaluation? Give an example. b. What are the reasons that would justify switching from normal order to applicative order evaluation? Give an example.

A: a. In normal order, expressions that would loop infinitely in applicative order can be terminated. Furthermore, values are only computed when called upon. So this function, which would crash in applicative order because we cannot divide by 0, would not crash because the action of division by zero is avoided by our boolean condition.

```
(define x 0)
(
  (lambda (a b c) (if a b c))
  (= x 0) "Cannot divide by zero" (/ 4 x)
)
```

In applicative order, all three arguments are evaluated before applying the lambda, so `(/ 4 0)` causes a division by zero error. In normal order, the arguments are substituted unevaluated into the body, and `if` only evaluates the consequent `"Cannot divide by zero"` since the condition is `#t`.

b. In applicative order, each argument is evaluated **once** before being passed. In normal order, arguments are substituted unevaluated and may be computed **multiple times** if referenced more than once in the body, leading to redundant computation.

```
(
  (lambda (x) (+ x x))
  (* 2 3)
)
```

In applicative order: `(* 2 3)` is computed once to 6, then `(+ 6 6) → 12`. In normal order: `(* 2 3)` is substituted twice $\rightarrow (+ (* 2 3) (* 2 3))$, so the multiplication is computed **twice**. Applicative order is more efficient here.

Question 1.10

Q1.10 a. In L3, what is the role of the function `valueToLitExp`? b. The `valueToLitExp` function is not needed in the normal evaluation strategy interpreter (`L3-normal.ts`). Why? c. The `valueToLitExp` function is not needed in the environment-model interpreter. Why?

A:

a. In the substitution model (applicative order), `applyClosure` evaluates the arguments to **Values** and then substitutes them into the procedure **body**. However, the body is composed of **CExp** (AST nodes), not Values. `valueToLitExp` converts a computed Value back into a corresponding CExp (e.g., `3 → NumExp(3)`, `#t → BoolExp(true)`) so that the substitution is type-compatible.

b. In normal order evaluation, arguments are **not evaluated** before substitution — the unevaluated expressions (which are already `CExp`) are substituted directly into the body. Since no `Values` are produced at substitution time, there is no type mismatch and no need for `valueToLitExp`.

c. In the environment model, values are never substituted into the body at all. Instead, they are stored in **frames** (an environment data structure) and looked up by variable name during evaluation. Since the body AST is never modified, `valueToLitExp` is redundant.

Question 1.11

Q1.11 a. Why is renaming not required in the environment model? b. Is renaming required in the substitution model for a case where the term that is substituted is "closed", i.e., does not contain free variables? Explain.

A:

a. In the environment model, there is no substitution at all — variables are looked up by navigating the frame hierarchy at runtime. Each frame keeps its own bindings, and lexical scoping ensures that lookup always follows the chain from the closure's defining environment. Since values are never substituted into the body AST, there is no opportunity for variable capture, and renaming is unnecessary.

b. No. Renaming is only needed to prevent **variable capture**, which occurs when a free variable in the substituted term is accidentally "caught" by a parameter of the same name in an enclosing lambda. If the substituted term is closed (contains no free variables), all its variables are already bound internally, so no outer lambda parameter can capture them. Therefore, renaming is not required.

For example, substituting the closed term `(lambda (w) (+ w 3))` into `(lambda (z) (f z))` is safe — there is no free variable that `z` could capture. However, substituting the open term `(lambda (w) (+ w z))` (where `z` is free) into the same expression would cause `z` to be incorrectly captured by the parameter `z`, which is why renaming is needed in that case.